

MSW + Expo + Jest

Project Playbook

A practical reference for Mock Service Worker, Expo / React Native, and Jest testing

QUICK PURPOSE

Learn MSW from scratch while testing a React Native User Directory against mocked GET and POST APIs.

Project Overview

A **User Directory** app used to learn MSW (Mock Service Worker) from scratch.

Stack

backend/ → Express + TypeScript (GET /users, GET /users/:id, POST /users)
frontend/ → Expo (React Native) + TypeScript + MSW + Jest

Final File Structure

```
msw-practice/  
  ■■■ backend/  
    ■ ■■■ src/  
      ■ ■■■ data.ts          ← in-memory user data  
      ■ ■■■ index.ts        ← Express server  
      ■ ■■■ .env             ← SERVER_IP, SERVER_PORT  
      ■ ■■■ package.json  
    ■■■ frontend/  
      ■■■ src/  
        ■ ■■■ mocks/  
          ■ ■■■ handlers.ts  ← shared MSW handlers (GET + POST)  
          ■ ■■■ server.ts    ← msw/native (app runtime, dev only)  
          ■ ■■■ server.node.ts ← msw/node (Jest only)  
          ■ ■■■ polyfills.ts ← Hermes polyfills (app runtime only)  
          ■ ■■■ setupTests.ts ← Jest lifecycle setup  
          ■ ■■■ __tests__/  
            ■ ■■■ App.test.tsx ← GET, loading, error tests  
            ■ ■■■ UserForm.test.tsx ← POST, body capture, 401, network failure tests  
          ■■■ app.config.ts    ← Expo config with dotenv  
          ■■■ config.ts        ← serverUrl helper  
          ■■■ App.tsx          ← main component  
          ■■■ babel.config.js  
          ■■■ .env             ← EXPO_PUBLIC_SERVER_IP, EXPO_PUBLIC_SERVER_PORT  
          ■■■ package.json
```

Key Versions

Versions from `package.json`:

Package	Version
expo	~56.0.11
react	19.2.3
react-native	0.85.3
msw	^2.3.5
jest	^29.7.0
jest-expo	^56.0.5
@testing-library/react-native	^14.0.1
react-test-renderer	^19.2.3
typescript	~6.0.3

Compatibility rule: jest-expo should match the Expo SDK major version. Expo SDK 56 → jest-expo@56.

Core Files

src/mocks/handlers.ts

```
import { http } from "msw";

const mockUsers = [
  { id: 1, name: "MSW Alice", email: "msw-alice@mock.com", role: "admin" },
  { id: 2, name: "MSW Bob", email: "msw-bob@mock.com", role: "member" },
  { id: 3, name: "MSW Carol", email: "msw-carol@mock.com", role: "member" },
];

export const handlers = [
  http.get("http://*/users", () => {
    console.log("[MSW] Intercepted GET /users");
    return new Response(JSON.stringify(mockUsers), {
      status: 200,
      headers: { "Content-Type": "application/json" },
    });
  }),

  http.post("http://*/users", async ({ request }) => {
    console.log("[MSW] Intercepted POST /users");
    const body = await request.json() as { name: string; email: string; role: string };
    const newUser = { id: mockUsers.length + 1, ...body };
    return new Response(JSON.stringify(newUser), {
      status: 201,
      headers: { "Content-Type": "application/json" },
    });
  }),
];
```

Why Response instead of HttpResponse.json? Use new `Response(JSON.stringify(...))`. The latter uses `ReadableStream` internally, which can break with Hermes and some Jest setups.

src/mocks/server.ts — app runtime

```
import { setupServer } from "msw/native";
import { handlers } from "../handlers";

export const server = setupServer(...handlers);
```

src/mocks/server.node.ts — Jest

```
import { setupServer } from "msw/node";
import { handlers } from "../handlers";

export const server = setupServer(...handlers);
```

src/setupTests.ts

```
const { config } = require("dotenv");
config();

// Required for React 19 – tells React this is a test environment
(globalThis as any).IS_REACT_ACT_ENVIRONMENT = true;

import { server } from "../mocks/server.node";

beforeAll(() => server.listen({ onUnhandledRequest: "warn" }));
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

App.tsx — MSW bootstrap

```
// Only start the native mock server in dev, never in Jest
if (__DEV__ && process.env.NODE_ENV !== "test") {
  require("../src/mocks/polyfills");
  const { server } = require("../src/mocks/server");
  server.listen({
    onUnhandledRequest(request: Request) {
      if (request.url.includes(":8081")) return; // ignore Expo internals
      console.warn(`[MSW] Unhandled: ${request.method} ${request.url}`);
    },
  });
  console.log("[MSW] Mock server started");
}
```

jest config in package.json

```
"jest": {
  "preset": "jest-expo",
  "setupFilesAfterEnv": ["../src/setupTests.ts"],
  "testEnvironmentOptions": {
    "customExportConditions": ["node", "require", "default"]
  },
  "moduleNameMapper": {
    "^msw/node$": "<rootDir>/node_modules/msw/lib/node/index.js"
  },
  "transformIgnorePatterns": [
    "node_modules/(?!(jest-)?react-native|@react-native(-community)?|.*react-native/.expo.*|expo(nent)?/.expo-google-fonts|react-navigation|@react-navigation/.unimodules/.unimodules|sentry-expo|native-base|react-native-svg|msw|@mswjs/.web-streams-polyfill|text-encoding)"
  ]
}
```

Testing Patterns

The golden rule

```
await render(<App />) // always await – flushes all async effects
screen.getByText("MSW Alice") // assert directly after, no waitFor needed

// Only use waitFor when an interaction triggers a NEW async update
await user.press(screen.getByTestId("btn-add"));
await waitFor(() => expect(screen.getByText("Dave New")).toBeTruthy());
```

Use userEvent with React 19

```
const user = userEvent.setup();

// ✓ async, properly waits for state to settle between interactions
await user.type(screen.getByTestId("input-name"), "Dave New");
await user.press(screen.getByTestId("btn-add"));

// ✗ synchronous, misses state updates in React 19 concurrent mode
fireEvent.changeText(screen.getByTestId("input-name"), "Dave New");
```

Per-test overrides

```
// Override for this test only – resetHandlers() cleans up automatically after
server.use(
  http.get("http://users", () => new Response(null, { status: 500 }))
);
```

Testing patterns reference

```
// Success state
await render(<App />);
expect(screen.getByText("MSW Alice")).toBeTruthy();

// Loading state (never-resolving handler)
server.use(http.get("http://*/users", () => new Promise(() => {})));
await render(<App />);
expect(screen.getByTestId("loading-indicator")).toBeTruthy();

// HTTP error (500, 401, etc.)
server.use(http.get("http://*/users", () => new Response(null, { status: 500 })));
await render(<App />);
expect(screen.getByText("Failed to fetch users")).toBeTruthy();

// Network failure
server.use(http.get("http://*/users", () => { throw new TypeError("Network request failed"); }));
await render(<App />);
expect(screen.getByText("Failed to fetch users")).toBeTruthy();

// Capture request body
let capturedBody: any = null;
server.use(http.post("http://*/users", async ({ request }) => {
  capturedBody = await request.json();
  return new Response(JSON.stringify({ id: 99, ...capturedBody }), { status: 201 });
}));
await render(<App />);
await user.type(screen.getByTestId("input-name"), "Eve Test");
await user.press(screen.getByTestId("btn-add"));
expect(capturedBody).toEqual({ name: "Eve Test", email: "eve@example.com", role: "member" });
```

Errors Encountered & Solutions

01. ``Failed to fetch users`` — *localhost not reachable on physical device*

Cause: localhost on a phone refers to the phone itself, not your computer.

Fix:

```
// frontend/.env
EXP0_PUBLIC_SERVER_IP=192.168.x.x // your actual IP from ipconfig/ifconfig
EXP0_PUBLIC_SERVER_PORT=3000
```

02. `MSW `async_hooks` error` — *wrong MSW entrypoint for React Native*

Cause: ``msw/node`` imports Node built-ins (``async_hooks``) unavailable in Hermes.

Fix:

```
msw/native → Expo Go / Hermes runtime
msw/node → Jest / real Node.js
msw/browser → web browsers
```

03. ``Property 'MessageEvent' doesn't exist`` + *other Hermes polyfill errors*

Cause: Hermes is stripped-down and missing Web APIs that MSW v2 needs. Define ``EventTarget``, ``Event``, ``MessageEvent``, ``BroadcastChannel``, ``TextEncoder``, ``TextDecoder``, and ``ReadableStream`` in ``src/mocks/polyfills.ts`` before MSW starts.

Fix:

```
npm install text-encoding web-streams-polyfill
```

04. ``JSON Parse error: Unexpected end of input`` — *empty mock response body*

Cause: ``HttpResponse.json()`` uses ``ReadableStream`` internally, which can break with Hermes polyfills.

Fix:

```
return new Response(JSON.stringify(data), {
  status: 200,
  headers: { "Content-Type": "application/json" },
});
```

05. `Cannot use import statement outside a module` — MSW ESM not transformed

Cause: Jest does not transform `node_modules` by default, while MSW and related packages ship ESM. Whitelist them in `transformIgnorePatterns` and map `msw/node` to its compiled output.

Fix:

```
"moduleNameMapper": {
  "^msw/node$": "<rootDir>/node_modules/msw/lib/node/index.js"
},
"transformIgnorePatterns": [
  "node_modules/(?!(jest-)?react-native|...|msw|@mswjs/.*)"
]
```

06. `rettime` ESM-only package breaking Jest

Cause: MSW 2.7+ introduced `rettime` as a dependency; it ships only `.mjs` with no CJS fallback, which can break Babel-based Jest.

Fix:

```
npm uninstall msw
npm install -D msw@2.3.5
```

07. `jest-expo` version mismatch

Cause: `jest-expo@57` requires `@react-native/jest-preset@^0.86.0`, while React Native 0.85.3 ships the 0.85.3 preset.

Fix:

```
npm install -D jest-expo@56
```

08. `@testing-library/jest-native` peer dependency conflict

Cause: `@testing-library/jest-native` is deprecated; its matchers were merged into `@testing-library/react-native` v12+. Do not install it or reference it in setup.

09. `react-test-renderer` version mismatch

Cause: It must exactly match the installed React version.

Fix:

```
npm install -D react-test-renderer@19.2.3
```

10. `setupFilesAfterFramework` typo

Cause: The correct Jest config key is `setupFilesAfterEnv`.

Fix:

```
"setupFilesAfterEnv": ["/src/setupTests.ts"]
```

11. `Cannot find name 'describe' / 'expect' / 'it'`

Cause: TypeScript does not know about Jest globals.

Fix:

```
npm install -D @types/jest

// tsconfig.json
"compilerOptions": { "types": ["jest"] }
```

12. Two MSW servers conflicting in Jest

Cause: `App.tsx` was starting `msw/native` during Jest, so `server.use()` overrides on the Node server had no effect.

Fix:

```
if (__DEV__ && process.env.NODE_ENV !== "test") {
  // start msw/native only outside Jest
}
```

13. `render` function has not been called / `screen` not bound

Cause: React 19 requires `IS_REACT_ACT_ENVIRONMENT = true` in the test environment.

Fix:

```
(globalThis as any).IS_REACT_ACT_ENVIRONMENT = true;
```

14. `fireEvent` missing state updates / overlapping `act()` warnings

Cause: `fireEvent` is synchronous and does not integrate reliably with React 19 concurrent mode. Replace it with `userEvent`.

Fix:

```
const user = userEvent.setup();
await user.type(screen.getByTestId("input-name"), "Dave New");
await user.press(screen.getByTestId("btn-add"));
```

MSW Entrypoints — Quick Reference

Environment	Import from	Used in
Expo Go / Hermes	msw/native	server.ts (app runtime)
Jest / Node.js	msw/node	server.node.ts (tests)
Browser	msw/browser	web apps

Option	When it runs	Use for
setupFiles	Before Jest globals exist	Polyfills, env vars, global patches
setupFilesAfterEnv	After Jest globals exist	beforeAll, afterEach, MSW lifecycle